

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT - CO2003

BÀI TẬP LỚN 2

CẢI THIẾN ỨNG DỤNG BotKify SỬ DỤNG CÂY AVL

Thành phố Hồ Chí Minh, 03/2026

ĐẶC TẢ BÀI TẬP LỚN

Phiên bản 1.0

1 Mục tiêu bài tập

Sau khi hoàn thành bài tập lớn này, sinh viên sẽ có khả năng:

- Áp dụng thành thạo các nguyên lý lập trình hướng đối tượng (OOP) thông qua thiết kế lớp nâng cao, kế thừa và đa hình.
- Hiểu, thiết kế và hiện thực cấu trúc dữ liệu cây tự cân bằng, cụ thể là AVL Tree và biến thể Threaded AVL Tree.
- Áp dụng các cấu trúc dữ liệu dạng cây nâng cao để nâng cấp và tối ưu hóa một hệ thống thực tế (quản lý danh sách phát cho ứng dụng BotKify), giúp giảm đáng kể độ phức tạp tính toán.

2 Giới thiệu

Trong BTL 1, sinh viên đã hiện thực thành công các tính năng cơ bản của một ứng dụng nghe nhạc trực tuyến có tên BotKify. Hệ thống hỗ trợ thêm, xóa và truy vấn bài hát trong danh sách phát dựa trên cấu trúc dữ liệu danh sách. Mặc dù hiệu quả với tập dữ liệu nhỏ, nhưng các cấu trúc tuyến tính bộc lộ những hạn chế nghiêm trọng về độ phức tạp tính toán khi số lượng bài hát tăng lên, thường yêu cầu thời gian $O(n)$ cho các thao tác tìm kiếm, chèn và xóa.

Trong bài tập này, sinh viên được yêu cầu nâng cấp ứng dụng BotKify để xử lý hiệu quả các tập dữ liệu lớn. Để đạt được điều này, các danh sách tuyến tính sẽ được thay thế bằng một cấu trúc dữ liệu nâng cao hơn: **AVL Tree** dạng key-value. Ứng dụng nâng cấp sẽ quản lý dữ liệu danh sách phát đồng thời đảm bảo nghiêm ngặt rằng các thao tác chèn, xóa và tìm kiếm được tối ưu hóa về độ phức tạp thời gian $O(\log n)$.

Hệ thống BotKify nâng cấp phải hỗ trợ các chức năng cốt lõi sau:

- **Quản lý bài hát:** Chèn, xóa và truy xuất bài hát trong danh sách phát. Bài hát được lưu trữ theo thứ tự đã sắp xếp (theo thứ tự bảng chữ cái theo tiêu đề bài hát, với ID bài hát đóng vai trò tie-breaker) sử dụng AVL Tree.
- **Phát liên tục:** Người dùng có thể chuyển sang bài hát tiếp theo hoặc trước đó một cách mượt mà trong danh sách phát dựa trên thứ tự bảng chữ cái.

- **Thao tác nâng cao:** Tính tổng điểm danh sách phát, so sánh các danh sách phát khác nhau, phát bài ngẫu nhiên và điều hướng phát nhạc xấp xỉ.

Trong một AVL tree tiêu chuẩn, việc tìm predecessor hoặc successor theo thứ tự in-order cho tính năng phát liên tục tốn $O(\log n)$ thời gian. Sinh viên được khuyến khích mở rộng hiện thực thành **Threaded AVL Tree** — một biến thể trong đó con trỏ **next** và **prev** được thêm vào mỗi nút để kết nối chúng thành một danh sách liên kết đôi theo thứ tự duyệt In-order. Cấu trúc này cho phép ứng dụng truy xuất bài hát tiếp theo hoặc trước đó với độ phức tạp thời gian $O(1)$.

3 AVL Tree và Threaded AVL Tree

Trước khi giới thiệu AVL Tree, hãy cùng ôn lại cây nhị phân tìm kiếm (BST). Cây nhị phân tìm kiếm (BST) là một cấu trúc dữ liệu phân cấp gồm các nút, trong đó mỗi nút có tối đa hai con, gọi là con trái và con phải. Trong một BST, con trái của một nút chứa giá trị nhỏ hơn giá trị của nút đó, và con phải chứa giá trị lớn hơn hoặc bằng.

Mặc dù BST giúp các thao tác tìm kiếm, chèn và xóa hiệu quả với độ phức tạp thời gian $O(\log n)$ cho cây cân bằng, nhưng có thể xuống cấp thành $O(n)$ nếu dữ liệu được chèn đã được sắp xếp, dẫn đến cây lệch. Để đảm bảo các thao tác $O(\log n)$ trong mọi trường hợp, chúng ta sử dụng BST tự cân bằng gọi là AVL Tree.

3.1 AVL Tree

AVL tree là một cây nhị phân tìm kiếm tự cân bằng. Trong một AVL tree, độ lệch chiều cao giữa hai cây con (trái và phải) của bất kỳ nút nào không được vượt quá một. Nếu ở bất kỳ thời điểm nào sự khác biệt này lớn hơn một, cây sẽ tự động thực hiện quá trình cân bằng lại để khôi phục thuộc tính này.

Để duy trì trạng thái cân bằng nghiêm ngặt này, bất cứ khi nào thao tác chèn hoặc xóa làm mất cân bằng cấu trúc, AVL tree sử dụng các phép quay cơ bản như quay trái, quay phải, hoặc quay kép. Nhờ cơ chế tự điều chỉnh này, tất cả các thao tác tìm kiếm, chèn và xóa trên AVL tree đều được đảm bảo đạt độ phức tạp thời gian tối ưu $O(\log n)$ trong cả trường hợp trung bình và trường hợp xấu nhất.

Sinh viên được khuyến khích tham khảo tài liệu bài giảng để hiện thực AVL tree.

3.2 Threaded AVL Tree

Trước khi giới thiệu Threaded AVL Tree, hãy ôn lại cách duyệt AVL Tree tiêu chuẩn. Như đã trình bày trong các phần trước, AVL Tree đảm bảo rằng các thao tác tìm kiếm, chèn và xóa duy trì độ phức tạp thời gian $O(\log n)$. Tuy nhiên, khi cần tìm predecessor hoặc successor theo in-order của một nút — một thao tác cơ bản cho tính năng phát liên tục trong danh sách phát — chúng ta phải duyệt lên và xuống các nhánh cây. Trong trường hợp xấu nhất, việc duyệt này cũng tốn $O(\log n)$ thời gian.

"Thread" trong Threaded AVL Tree đại diện cho một liên kết trực tiếp giữa một nút và predecessor hoặc successor theo in-order của nó. Trong khi một nút AVL tree tiêu chuẩn chỉ chứa con trỏ con trái và phải để biểu diễn cấu trúc phân cấp, một nút Threaded AVL Tree chứa thêm hai con trỏ: **prev** và **next**. Các con trỏ này được thiết kế để xử lý hiệu quả việc duyệt dữ liệu tuần tự.

Trong một Threaded AVL tree, mỗi nút đại diện cho một bài hát trong danh sách phát. Giống như AVL Tree tiêu chuẩn, mỗi nút có vị trí phân cấp được xác định bởi khóa của nó (ví dụ: tên bài hát). Nhưng ngoài ra, con trỏ **prev** liên kết trực tiếp đến nút ngay trước nó theo thứ tự bảng chữ cái, và con trỏ **next** liên kết trực tiếp đến nút ngay sau nó. Điều này tạo ra một danh sách liên kết đôi được nhúng trực tiếp trong cấu trúc cây.

Để dễ hiểu, hãy tham khảo ví dụ sau. Hãy chèn tất cả các ID bài hát được liệt kê vào Threaded AVL Tree theo thứ tự này:

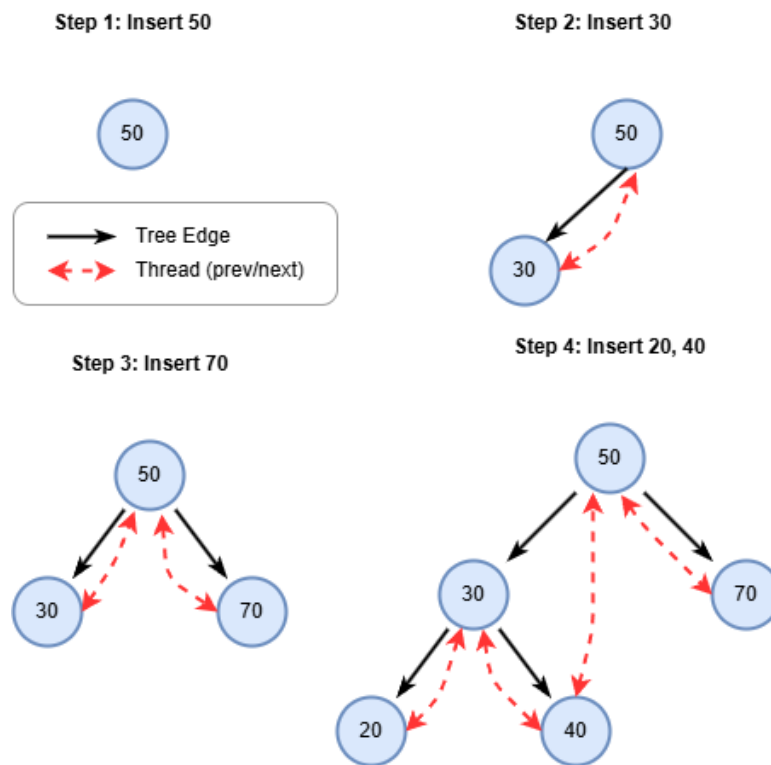
50, 30, 70, 20, 40

Nút gốc là 50. Vì là nút duy nhất, con trỏ **prev** và **next** của nó đều là NULL. Nó đóng vai trò vừa là đầu vừa là đuôi của danh sách thread.

Khi chèn 30, nó trở thành con trái của 50. Trong duyệt in-order, 30 đến trước 50. Do đó, con trỏ **next** của 30 trở đến 50, và con trỏ **prev** của 50 trở ngược lại 30. Nút 30 trở thành đầu mới của danh sách thread.

Khi chèn 70, nó trở thành con phải của 50. Trong chuỗi in-order, 70 đến sau 50. Do đó, con trỏ **next** của 50 trở đến 70, và con trỏ **prev** của 70 trở đến 50. Nút 70 trở thành đuôi mới.

Tiếp tục quá trình này với 20 và 40, cuối cùng chúng ta có cây hoàn chỉnh, trong đó các cạnh phân cấp duy trì thuộc tính BST cho tìm kiếm $O(\log n)$, và các cạnh thread duy trì một chuỗi tuyến tính hoàn hảo: 20 <-> 30 <-> 40 <-> 50 <-> 70.



Hình 1: Chèn từng bước vào Threaded AVL tree

3.2.1 Thuật toán Chèn

Việc chèn vào Threaded AVL tree hoạt động giống hệt như AVL tree tiêu chuẩn, với điều chỉnh rằng sau khi chèn theo cấu trúc phân cấp, chúng ta phải kết nối nút mới vào danh sách liên kết đôi.

Thuật toán duy trì thread trong quá trình chèn tiến hành như sau:

1. Duyệt xuống cây để tìm điểm chèn đúng như thông thường.
2. Trong quá trình duyệt, theo dõi predecessor (**pred**) và successor (**succ**) theo in-order của vị trí sẽ đặt nút mới.
3. Tạo nút mới.
4. Kết nối thread: Đặt **prev** của nút mới thành **pred**, và **next** thành **succ**.
5. Cập nhật các nút lân cận.
6. Lăn ngược về gốc, cập nhật các hệ số cân bằng và thực hiện phép quay AVL nếu cần.

Ví dụ 3.1

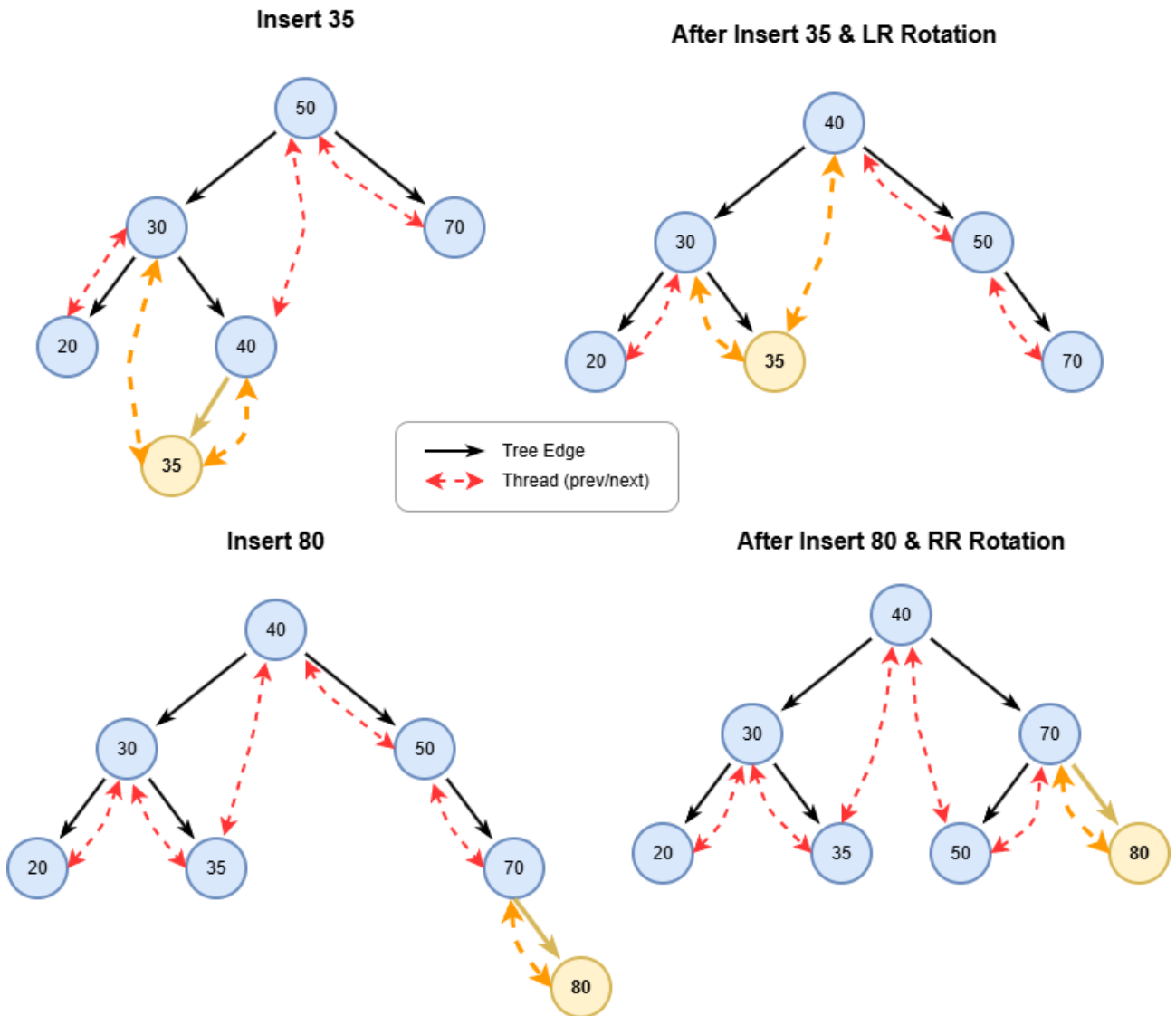
Hãy tiếp tục từ Threaded AVL tree trước đó với chuỗi thread: 20 $\leftrightarrow$ 30 $\leftrightarrow$ 40 $\leftrightarrow$ 50 $\leftrightarrow$ 70. Gốc của cây là 50.

Chèn 35

- **Chèn & Kết nối:** 35 được chèn làm con trái của 40. Chúng ta kết nối nó theo logic giữa predecessor (30) và successor (40) trong danh sách thread: 30 $\leftrightarrow$ 35 $\leftrightarrow$ 40.
- **Phép quay AVL:** Việc chèn gây mất cân bằng tại gốc 50 (trường hợp LR). Thực hiện **Phép quay Trái-Phải**, khiến 40 trở thành gốc mới.

Chèn 80

- **Chèn & Kết nối:** Từ gốc mới 40, 80 được chèn làm con phải của 70. Chúng ta nối nó vào cuối danh sách thread: 70 $\leftrightarrow$ 80.
- **Phép quay AVL:** Nút 50 mất cân bằng (trường hợp RR). Thực hiện **Phép quay Trái**, kéo 70 lên làm cha mới của 50.



Hình 2: Chèn từng bước 35 và 80 vào Threaded AVL tree

3.2.2 Thuật toán Xóa

Việc xóa trong Threaded AVL Tree yêu cầu xác định chính xác **nút** đang bị xóa khỏi bộ nhớ, vì đây là nút cần được bỏ qua trong danh sách liên kết đôi.

Quá trình xóa của chúng ta sẽ hoạt động như sau. Quá trình này là đệ quy, giả sử nút cần xóa là u :

- Nếu u có 0 hoặc 1 con:** Chính nút u sẽ bị xóa khỏi cây. Trước khi xóa u , chúng ta phải bỏ qua nó trong danh sách thread. Gọi P là **prev** của u và S là **next** của u . Chúng ta liên kết P và S trực tiếp với nhau ($P \leftrightarrow S$), hoàn toàn bỏ qua u . Sau đó, tiến hành xóa và cân bằng AVL tiêu chuẩn.

- (b) **Nếu u có 2 con:** Chúng ta không thể xóa u . Thay vào đó, chúng ta tìm successor theo in-order r của nó (nút nhỏ nhất trong cây con phải của nó). Chúng ta sao chép dữ liệu (thông tin bài hát) từ r vào u . Tại thời điểm này, nút u thực sự trở thành r về mặt dữ liệu. Nút thực sự bị xóa khỏi bộ nhớ cây là r . Do đó, chúng ta phải kết nối lại các thread để bỏ qua nút r , kết nối **prev** của r trực tiếp với **next** của r . Sau khi kết nối lại, chúng ta đệ quy xóa nút cũ r .

Ví dụ 3.2

Xét một Threaded AVL Tree với chuỗi: 20 $\leftrightarrow$ 30 $\leftrightarrow$ 40 $\leftrightarrow$ 50 $\leftrightarrow$ 60 $\leftrightarrow$ 70 $\leftrightarrow$ 80 $\leftrightarrow$ 90. Gốc của cây là 50.

- **Trạng thái ban đầu:** Con trái của nút 50 là 30. Con phải của nó là 70. Nút 70 có con trái (60) và con phải (80).
- **Mục tiêu:** Chúng ta muốn xóa nút gốc 50 (có 2 con).

Bước 1: Tìm Successor và Sao chép Dữ liệu

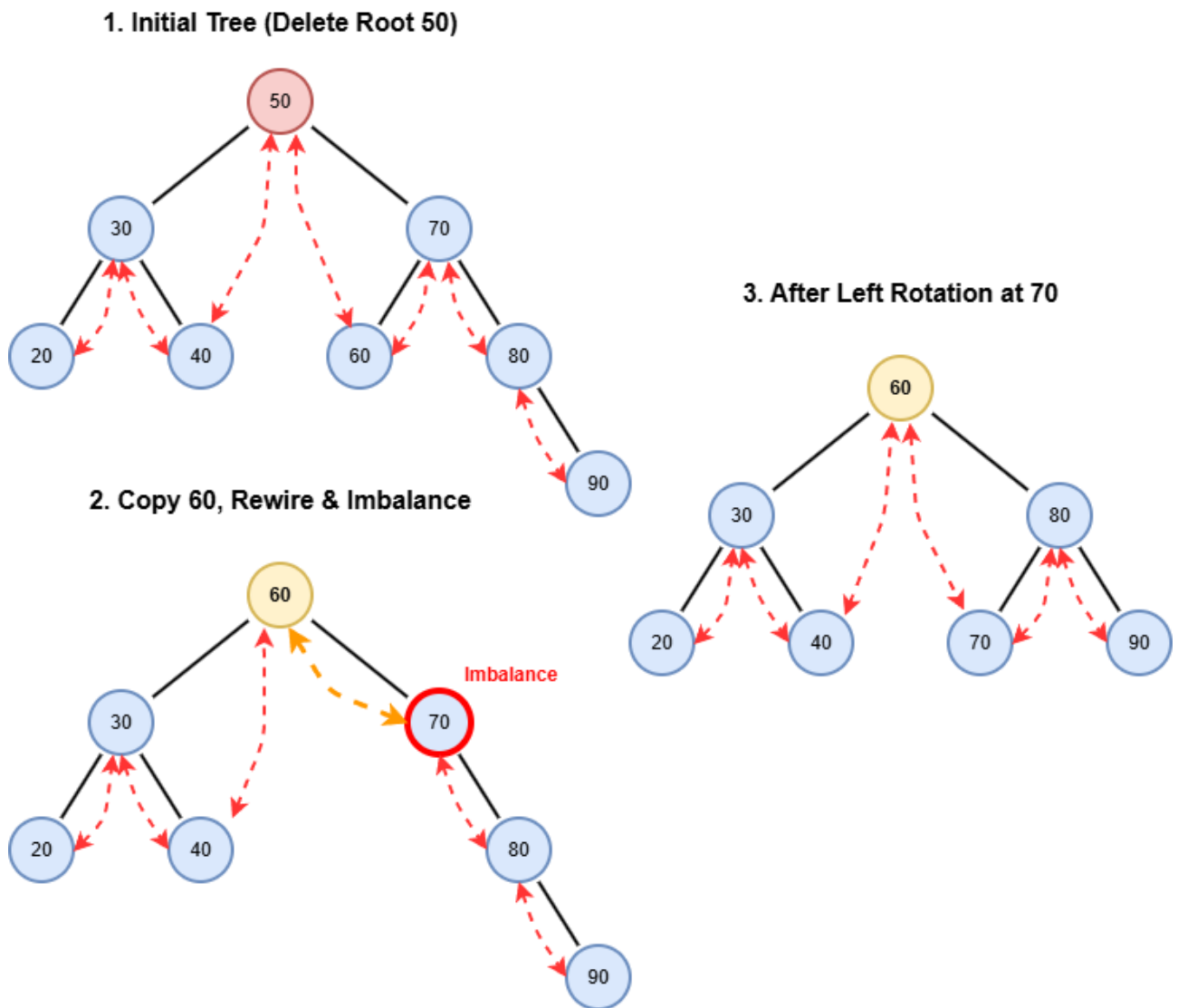
1. Successor theo in-order của 50 là 60 (nút nhỏ nhất trong cây con phải của 50).
2. Chúng ta sao chép dữ liệu '60' vào nút gốc. Nút gốc không còn là 50 nữa; nó bây giờ giữ 60.
3. Nút 60 ban đầu (là con trái của 70) là nút sẽ bị xóa vĩnh viễn khỏi bộ nhớ cây.

Bước 2: Kết nối lại Thread (Bỏ qua nút bị xóa)

1. Trước khi giải phóng nút 60, chúng ta phải bỏ qua nó trong danh sách liên kết đôi.
2. Predecessor của nút 60 là nút gốc (bây giờ giữ '60'). Successor của nó là 70.
3. Chúng ta liên kết chúng trực tiếp: $\text{Root} \rightarrow \text{next} = 70$ và $70 \rightarrow \text{prev} = \text{Root}$. Chuỗi cập nhật hoàn hảo thành ... $\leftrightarrow$ 40 $\leftrightarrow$ 60 $\leftrightarrow$ 70 $\leftrightarrow$ 80 $\leftrightarrow$ 90.

Bước 3: Cân bằng AVL và Phép quay

1. Chúng ta xóa nút 60 ban đầu.
2. Chúng ta lần ngược lên để cập nhật các hệ số cân bằng. Nút 70 đã mất con trái. Chiều cao cây con trái của nó bây giờ là 0, nhưng cây con phải (gốc tại 80) có chiều cao 2.
3. **Mất cân bằng!** Hệ số cân bằng của nút 70 trở thành trường hợp RR.
4. Thực hiện **Phép quay Trái** tại nút 70. Nút 80 di chuyển lên để trở thành con phải mới của gốc. Nút 70 di chuyển xuống để trở thành con trái của 80.



Hình 3: Xóa từng bước trong Threaded AVL tree

4 Yêu cầu

Trong bài tập này, sinh viên được yêu cầu hiện thực các lớp sau để mô tả quá trình xử lý dữ liệu và hiện thực ứng dụng nghe nhạc trực tuyến sử dụng cấu trúc AVL Tree, bao gồm interface IBST, lớp AVL, lớp ThreadedAVL, lớp Song, và lớp Playlist.

4.1 Interface IBST

Lớp IBST là một interface trừu tượng định nghĩa các thao tác tiêu chuẩn cho cây nhị phân tìm kiếm dạng key-value.

Mô tả cho lớp IBST như sau:

```
1 template <typename K, typename V>
2 class IBST {
3 public:
4     virtual ~IBST() = default;
5     virtual bool insert(const K& key, const V& value) = 0;
6     virtual bool erase(const K& key) = 0;
7     virtual V* find(const K& key) = 0;
8     virtual bool contains(const K& key) const = 0;
9     virtual int size() const = 0;
10    virtual bool empty() const = 0;
11    virtual void clear() = 0;
12    virtual int height() const = 0;
13    virtual std::list<K> ascendingList() = 0;
14    virtual std::list<K> descendingList() = 0;
15    virtual std::string toString(
16        std::string (*entry2str)(const K&, const V&) = nullptr
17    ) const = 0;
18 };
```

Vì đây là lớp trừu tượng thuần túy, sinh viên không cần hiện thực trực tiếp các phương thức này trong IBST, nhưng phải override và hiện thực tất cả chúng trong lớp dẫn xuất AVL.

4.2 Lớp AVL (4 điểm)

Lớp AVL kế thừa từ IBST và lưu trữ dữ liệu trong cấu trúc AVL tree tự cân bằng. Nó cung cấp cho người dùng các thao tác cân bằng đảm bảo độ phức tạp thời gian $O(\log n)$.

Mô tả cho lớp AVL như sau:

```
1 template <typename K, typename V>
2 class AVL : public IBST<K, V> {
3 public:
4     struct Node {
5         K key;
6         V value;
7         int bfactor;
8         Node* pLeft;
9         Node* pRight;
10        Node(const K& k, const V& v);
11        virtual ~Node() = default;
12        std::string toString(...) const;
13    };
14 protected:
15     Node* pRoot;
16     virtual void onInserted(Node* newNode, Node* pred, Node* succ) {}
17     virtual void onErasing(Node* delNode, Node* pred, Node* succ) {}
18     virtual void onReplaceBySuccessor(Node* target, Node* succ, Node*
succNext) {}
19     virtual Node* createNode(const K& key, const V& value) {
20         return new Node(key, value);
21     }
22 public:
23     AVL();
24     virtual ~AVL();
25     // Inherited methods from IBST...
26 };
```

Dưới đây là thông tin chi tiết về các phương thức và thuộc tính trong lớp AVL:

1. Node* pRoot;

- Nút gốc của AVL tree. Được khởi tạo bằng nullptr.

2. virtual Node* createNode(const K& key, const V& value);

- Phương thức factory để tạo một Node mới. Trả về con trỏ đến nút vừa được cấp phát.

3. virtual void onInserted(Node* newNode, Node* pred, Node* succ);

- Phương thức hook được gọi ngay sau khi newNode được chèn vào cây.
- pred và succ lần lượt là predecessor và successor theo in-order của newNode.
- Trong lớp AVL, phương thức này không làm gì. Nó được thiết kế để được override bởi ThreadedAVL.

4. `virtual void onErasing(Node* delNode, Node* pred, Node* succ);`

- Phương thức hook được gọi ngay trước khi một nút có 0 hoặc 1 con (`delNode`) bị xóa.
- Trong lớp AVL, phương thức này không làm gì.

5. `virtual void onReplaceBySuccessor(Node* target, Node* succ, Node* succNext);`

- Phương thức hook được gọi trong quá trình xóa nút `target` có 2 con. `succ` là successor theo in-order của `target`, và `succNext` là nút ngay sau `succ`.
- Trong lớp AVL, phương thức này không làm gì.

6. `bool insert(const K& key, const V& value);`

- Chèn một cặp key-value mới vào cây.
- Nếu `key` đã tồn tại, hàm không thêm vào và trả về `false`.
- Sau khi chèn, cây phải cập nhật các hệ số cân bằng và thực hiện phép quay (LL, RR, LR, RL) nếu cần để duy trì thuộc tính AVL.
- **Gợi ý:** Sinh viên phải xác định đúng `pred` và `succ` và gọi `onInserted` trước khi trả về `true`.

7. `bool erase(const K& key);`

- Xóa nút chứa `key` đã cho.
- Nếu không tìm thấy khóa, trả về `false`.
- Sau khi xóa, cây phải lần ngược về gốc, cập nhật các hệ số cân bằng và thực hiện phép quay để duy trì thuộc tính AVL.
- **Gợi ý:** Phải gọi `onErasing` hoặc `onReplaceBySuccessor` đúng cách tùy thuộc vào số con của nút bị xóa.

8. `V* find(const K& key);`

- Trả về con trỏ đến giá trị liên kết với `key`. Trả về `nullptr` nếu không tìm thấy khóa. Độ phức tạp thời gian phải là $O(\log n)$.

9. `std::list<K> ascendingList();`

- Duyệt cây theo In-order và trả về một `std::list` chứa tất cả các khóa theo thứ tự tăng dần.

10. `int height() const;`

- Trả về chiều cao hiện tại của cây. Nếu cây không có phần tử nào, chiều cao là 0.

4.3 Lớp ThreadedAVL (3 điểm)

Lớp ThreadedAVL mở rộng lớp AVL bằng cách thêm khả năng threading (danh sách liên kết đôi) để hỗ trợ duyệt in-order trong thời gian $O(1)$.

Mô tả cho lớp ThreadedAVL như sau:

```
1 template <typename K, typename V>
2 class ThreadedAVL : public AVL<K, V> {
3 private:
4     using Base = AVL<K, V>;
5     using AVLNode = typename Base::Node;
6     struct TNode : public Base::Node {
7         TNode* prev;
8         TNode* next;
9         TNode(const K& k, const V& v);
10    };
11    TNode* head_;
12    TNode* tail_;
13 public:
14     class Iterator {
15     public:
16         Iterator& operator++();
17         Iterator& operator--();
18         const K& key() const;
19         V& value();
20         bool isNull() const;
21     };
22     ThreadedAVL();
23     ~ThreadedAVL() override;
24
25     std::list<K> ascendingList() override;
26     std::list<K> descendingList() override;
27     void clear() override;
28
29     Iterator beginIt();
30     Iterator endIt();
31     Iterator rbeginIt();
32     Iterator findIt(const K& key);
33
34 protected:
35     AVLNode* createNode(const K& key, const V& value) override;
36     void onInserted(AVLNode* newNode, AVLNode* pred, AVLNode* succ) override
37     ;
```

```
37     void onErasing(AVLNode* delNode, AVLNode* pred, AVLNode* succ) override;  
38     void onReplaceBySuccessor(AVLNode* target, AVLNode* succ, AVLNode*  
succNext) override;  
39 };
```

Dưới đây là thông tin chi tiết về các phương thức trong lớp ThreadedAVL:

1. TNode* head_; và TNode* tail_;

- Con trỏ đến nút nhỏ nhất và lớn nhất trong cây, đại diện cho đầu và đuôi của danh sách liên kết đôi.

2. AVLNode* createNode(const K& key, const V& value);

- Override phương thức factory của lớp cơ sở để cấp phát động và trả về một TNode thay vì Node tiêu chuẩn.

3. void onInserted(...);, void onErasing(...);, void onReplaceBySuccessor(...);

- Sinh viên phải hiện thực logic để kết nối lại đúng cách các con trỏ prev và next của các nút bị ảnh hưởng nhằm duy trì chuỗi thread một cách hoàn hảo.

4. std::list<K> ascendingList();

- Override phép duyệt đệ quy. Phương thức này phải thu thập các khóa bằng cách lặp qua các con trỏ next bắt đầu từ head_.

5. Iterator operator++(); và Iterator operator-();

- Tiến iterator đến nút next hoặc prev trong đúng $O(1)$ thời gian.

4.4 Lớp Song

Lớp Song đại diện cho một bài hát trong danh sách phát.

Mô tả cho lớp Song như sau:

```
1 class Song {  
2 private:  
3     int id;  
4     std::string title;  
5     std::string artist;  
6     std::string album;  
7     int duration;  
8     int score;  
9     std::string url;
```

```
10     int play_count;
11 public:
12     Song(int id, std::string title, std::string artist, std::string album,
13         int duration, int score, std::string url);
14     std::string toString() const;
15 };
```

4.5 Lớp Playlist (3 điểm)

Lớp Playlist là mô-đun ứng dụng cốt lõi quản lý một tập hợp bài hát sử dụng AVL Tree bên dưới.

```
1 using SongKey = std::pair<std::string, int>; // (title, id)
2
3 class Playlist {
4 private:
5     std::string name;
6     int size;
7     int currentIndex;
8 #ifdef USE_THREADED_AVL
9     ThreadedAVL<SongKey, Song*> songs;
10    ThreadedAVL<SongKey, Song*>::Iterator currentIt;
11    bool hasCurrent;
12 #else
13    AVL<SongKey, Song*> songs;
14 #endif
15 private:
16    SongKey makeKey(Song* s) const;
17    void resetPlayback();
18 public:
19    Playlist(std::string name);
20    int getSize() const;
21    bool empty() const;
22    void clear();
23    void addSong(Song* s);
24    void removeSong(int index);
25    Song* getSong(int index) const;
26    Song* playNext();
27    Song* playPrevious();
28    int getTotalScore();
29    bool compareTo(const Playlist& p, int numSong);
30    void playRandom(int index);
```

```
31     int playApproximate(int step);  
32 };
```

Dưới đây là thông tin chi tiết về các thành viên dữ liệu và phương thức trong lớp `Playlist`:

1. `std::string name`;

- Tên của danh sách phát (playlist).

2. `int size`;

- Số lượng bài hát hiện có trong danh sách phát. Sinh viên cần cập nhật biến này mỗi khi thêm hoặc xóa bài hát.

3. `int currentIndex`;

- Chỉ số (index dựa trên 0) của bài hát đang được phát hiện tại theo chuỗi in-order đã sắp xếp.
- Nếu chưa có bài hát nào được phát, hoặc danh sách phát trống, biến này nên được thiết lập về một giá trị mặc định (ví dụ: -1).

4. `AVL<SongKey, Song*> songs`; (hoặc `ThreadedAVL`)

- Cấu trúc cây dùng để lưu trữ và quản lý các bài hát. Khóa (key) là `SongKey` và giá trị (value) là con trỏ `Song*`.

5. `Iterator currentIt`; và `bool hasCurrent`; (Chỉ áp dụng khi có `USE_THREADED_AVL`)

- `currentIt`: Bộ Iterator trỏ trực tiếp đến node của bài hát đang phát, hỗ trợ thao tác chuyển bài $O(1)$.
- `hasCurrent`: Cờ (flag) đánh dấu xem trạng thái phát nhạc hiện tại có hợp lệ hay không (hỗ trợ phân biệt khi iterator chưa được khởi tạo hoặc khi bài hát đang phát bị xóa).

6. `SongKey`

- Được định nghĩa là `std::pair<std::string, int>`. Bài hát được sắp xếp theo thứ tự bảng chữ cái theo `title`. Nếu hai bài hát có tiêu đề hoàn toàn giống nhau, `id` được sử dụng như tie-breaker để đảm bảo tính duy nhất trong AVL tree.

7. `void addSong(Song* s)`;

- Chèn con trỏ bài hát `s` vào AVL tree sử dụng `SongKey`. Tăng `size`.

8. `void removeSong(int index)`;

- Xóa bài hát tại `index` dựa trên 0 theo chuỗi in-order đã sắp xếp.

- Giảm `size`. Nếu bài hát đang phát bị xóa, trạng thái phát nhạc phải được reset đúng cách.

9. `Song* getSong(int index) const;`

- Trả về con trỏ đến bài hát tại `index` dựa trên 0 theo chuỗi in-order đã sắp xếp. Trả về `nullptr` nếu `index` nằm ngoài phạm vi.

10. `Song* playNext();`

- Tiến phát nhạc đến bài hát tiếp theo trong chuỗi đã sắp xếp và trả về con trỏ của nó.
- Nếu không có bài hát nào đang phát, bắt đầu từ bài hát đầu tiên. Nếu đang ở bài cuối, trả về `nullptr`.
- **Gợi ý:** Nếu biên dịch với `USE_THREADED_AVL`, phải sử dụng toán tử `++` của `Iterator` để đạt độ phức tạp thời gian $O(1)$.

11. `Song* playPrevious();`

- Tua lại phát nhạc về bài hát trước đó trong chuỗi đã sắp xếp và trả về con trỏ của nó. Trả về `nullptr` nếu đang ở bài đầu tiên.

12. `int getTotalScore();`

- Trả về tổng thuộc tính `score` của tất cả bài hát hiện có trong danh sách phát.

13. `bool compareTo(const Playlist& p, int numSong);`

- So sánh danh sách phát hiện tại với danh sách phát `p` dựa trên tổng điểm của `numSong` bài hát đầu tiên (theo chuỗi đã sắp xếp).
- Trả về `true` nếu điểm tính toán của danh sách phát hiện tại lớn hơn hoặc bằng điểm của `p`.

14. `void playRandom(int index);`

- Nhảy trạng thái phát nhạc trực tiếp đến bài hát tại `index` được chỉ định.

15. `int playApproximate(int step);`

- Từ vị trí phát nhạc hiện tại, nhảy về phía trước (nếu `step > 0`) hoặc phía sau (nếu `step < 0`) đúng `|step|` vị trí trong chuỗi đã sắp xếp.
- Trả về `currentIndex` mới.

5 Hướng dẫn

Để hoàn thành bài tập này, sinh viên phải:

1. Đọc kỹ toàn bộ file mô tả này.
2. Tải file initial.zip và giải nén. Sau khi giải nén, sinh viên sẽ nhận được các file sau: main.cpp, main.h, utils.h, IBST.h, AVL.h, ThreadedAVL.h, Playlist.h, và Playlist.cpp. Sinh viên chỉ cần nộp AVL.h, ThreadedAVL.h, và Playlist.cpp, vì vậy không nên chỉnh sửa các file main.h, IBST.h, và Playlist.h khi chạy chương trình.
3. Sinh viên nên sử dụng lệnh sau để biên dịch:

Chế độ cơ bản:

```
g++ -o main main.cpp Playlist.cpp -I . -std=c++17
```

Chế độ đầy đủ:

```
g++ -o main main.cpp Playlist.cpp -I . -std=c++17  
-DUSE_THREADED_AVL
```

Lệnh này được sử dụng trong command prompt/terminal để biên dịch chương trình. Nếu sinh viên sử dụng IDE để chạy chương trình, cần lưu ý những điều sau: thêm tất cả các file cần thiết vào project/workspace của IDE; thay đổi lệnh biên dịch của IDE cho phù hợp. IDE thường cung cấp các nút để biên dịch (Build) và chạy chương trình (Run). Khi nhấn Build, IDE sẽ thực thi lệnh biên dịch tương ứng, thường chỉ biên dịch main.cpp. Sinh viên cần cấu hình IDE để thay đổi lệnh biên dịch: thêm file Playlist.cpp, thêm các tùy chọn -std=c++17, -I ., và -DUSE_THREADED_AVL (nếu hiện thực phần Threaded AVL).

4. Chương trình sẽ được chấm điểm trên nền tảng UNIX. Nền tảng và trình biên dịch sinh viên sử dụng có thể khác với môi trường chấm điểm thực tế. Nền tảng nộp bài trên BKeL được thiết lập tương tự như môi trường chấm điểm thực tế. Sinh viên phải chạy chương trình trên nền tảng nộp bài và sửa các lỗi xảy ra trên BKeL để đảm bảo kết quả đúng trong khi chấm điểm thực tế.
5. Chỉnh sửa các file AVL.h, ThreadedAVL.h, và Playlist.cpp để hoàn thành bài tập này và đảm bảo các yêu cầu sau được đáp ứng:
 - Tất cả các phương thức được mô tả trong đặc tả này phải được hiện thực để quá trình biên dịch thành công. Nếu sinh viên không thể hiện thực một phương thức, hãy cung cấp hiện thực rỗng cho phương thức đó. Mỗi testcase sẽ gọi một số phương thức được mô tả để kiểm tra kết quả trả về.
 - Chỉ có 1 chỉ thị **include** trong file AVL.h, đó là **#include "IBST.h"**. Chỉ có 1 chỉ thị **include** trong file ThreadedAVL.h, đó là **#include "AVL.h"**. Chỉ có 1 chỉ thị **include** trong file Playlist.cpp, đó là **#include "Playlist.h"**. Ngoài ra, không có chỉ thị **#include** nào khác được phép trong các file này.

6. File main.cpp cung cấp một số testcase đơn giản trong các hàm có định dạng **"tc<x>()"**.
7. Sinh viên được phép viết thêm các phương thức và thuộc tính trong các lớp yêu cầu. Sinh

viên cũng được phép viết thêm các lớp ngoài các hiện thực yêu cầu.

8. Sinh viên được yêu cầu thiết kế và sử dụng các cấu trúc dữ liệu dựa trên các loại cây đã học.
9. Sinh viên phải giải phóng tất cả bộ nhớ được cấp phát động khi chương trình kết thúc.

6 Nộp bài

Sinh viên chỉ được nộp 3 file: AVL.h, ThreadedAVL.h, và Playlist.cpp, trước thời hạn được chỉ định trong đường dẫn "Assignment 2 - Submission". Có một số testcase đơn giản được sử dụng để kiểm tra kết quả của sinh viên nhằm đảm bảo rằng kết quả có thể biên dịch và chạy được. Sinh viên có thể nộp bài bao nhiêu lần tùy ý, nhưng chỉ lần nộp cuối cùng mới được chấm điểm. Vì hệ thống không thể xử lý tải khi quá nhiều sinh viên nộp bài cùng lúc, sinh viên nên nộp bài sớm nhất có thể. Sinh viên sẽ chịu rủi ro nếu nộp bài gần thời hạn. Sau khi thời hạn nộp bài đã qua, hệ thống sẽ đóng lại và sinh viên sẽ không thể nộp bài nữa. Các bài nộp qua phương thức khác sẽ không được chấp nhận.

7 Quy định khác

- Sinh viên phải hoàn thành bài tập này một mình và phải ngăn chặn người khác sao chép kết quả của mình. Nếu không tuân thủ, sinh viên sẽ bị kỷ luật theo chính sách gian lận của trường.
- Mọi quyết định của giảng viên phụ trách bài tập này là quyết định cuối cùng.
- Sinh viên không được cung cấp testcase sau khi chấm điểm, mà chỉ được cung cấp thông tin về chiến lược thiết kế testcase và phân phối số lượng sinh viên đúng theo từng testcase.
- Nội dung bài tập sẽ được **harmony** với một vài câu hỏi trong kỳ thi với nội dung tương tự.

8 Nhật ký thay đổi

—————KẾT THÚC—————